

Protokol profilingu

Popis vstupov

Program bol profilovaný s tromi vstupmi rôznej veľkosti, aby bolo možné sledovať správanie programu pri malej, strednej a veľkej záťaži. Boli použité vstupy o veľkosti: 10, 10^3 , 10^6 .

Pre 10 čísel bol celkový čas trvania programu 0.00134s

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.001198	0.001198	0.001342	0.001342	profiling.py:10(deviation)
1	6.574e-05	6.574e-05	6.574e-05	6.574e-05	~:0(<built-in method builtins.print>)
11	5.392e-06	4.902e-07	4.707e-05	4.279e-06	mathlib.py:72(pow)
11	4.168e-05	3.789e-06	4.168e-05	3.789e-06	~:0(<built-in method math.pow>)
3	1.656e-05	5.52e-06	2.257e-05	7.524e-06	<frozen codecs>:319(decode)
1	1.643e-05	1.643e-05	1.643e-05	1.643e-05	~:0(<method 'disable' of '_lsprof.Profiler' objects>)
3	6.01e-06	2.003e-06	6.01e-06	2.003e-06	~:0(<built-in method _codecs.utf_8_decode>)

Najviac času program strávil vo funkcii pow (umocňovanie).

Pre 1000 čísel bol celkový čas trvania programu 0.00233s

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.001831	0.001831	0.002334	0.002334	profiling.py:10(deviation)
1001	0.0001216	1.214e-07	0.0002478	2.475e-07	mathlib.py:72(pow)
1001	0.0001262	1.261e-07	0.0001262	1.261e-07	~:0(<built-in method math.pow>)
2000	0.0001077	5.384e-08	0.0001077	5.384e-08	mathlib.py:19(add)
1	6.738e-05	6.738e-05	6.738e-05	6.738e-05	~:0(<built-in method builtins.print>)
1	5.372e-05	5.372e-05	5.372e-05	5.372e-05	~:0(<method 'split' of 'str' objects>)
3	1.639e-05	5.463e-06	2.178e-05	7.26e-06	<frozen codecs>:319(decode)

Najviac času program strávil vo funkciách pow a add.

Pre 1000000 čísel bol celkový čas trvania programu 1.01s

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.6091	0.6091	1.013	1.013	profiling.py:10(deviation)
1000001	0.1221	1.221e-07	0.2196	2.196e-07	mathlib.py:72(pow)
2000000	0.1123	5.615e-08	0.1123	5.615e-08	mathlib.py:19(add)
1000001	0.09752	9.752e-08	0.09752	9.752e-08	~:0(<built-in method math.pow>)
1	0.06624	0.06624	0.06624	0.06624	~:0(<method 'split' of 'str' objects>)
358	0.0008318	2.323e-06	0.005707	1.594e-05	<frozen codecs>:319(decode)
358	0.004875	1.362e-05	0.004875	1.362e-05	~:0(<built-in method _codecs.utf_8_decode>)

Aj pri veľkom vstupe dominovali funkcie pow a add - predstavujú hlavný priestor pre optimalizáciu.

Po optimalizácii bol pre 1000000 čísel celkový čas trvania programu 0.804s

ncalls	⚡	tottime	⚡	percall	⚡	cumtime	⚡	percall	⚡	filename:lineno(function)
1		0.5726		0.5726		0.8042		0.8042		profiling.py:10(deviation)
2000000		0.1018		5.089e-08		0.1018		5.089e-08		mathlib.py:19(add)
1		0.0667		0.0667		0.0667		0.0667		~:0(<method 'split' of 'str' objects>)
1000001		0.05754		5.754e-08		0.05754		5.754e-08		mathlib.py:37(mul)
358		0.0008287		2.315e-06		0.005483		1.532e-05		<frozen codecs>:319(decode)
358		0.004655		1.3e-05		0.004655		1.3e-05		~:0(<built-in method _codecs.utf_8_decode>)
1		7.166e-05		7.166e-05		7.166e-05		7.166e-05		~:0(<built-in method builtins.print>)

Po optimalizácii došlo k zníženiu celkového času behu programu z 1.01 s na 0.804 s. Nahradením volaní `pow(x,2)` priamym násobením `x*x` sa odstránil hlavný bottleneck programu a výpočet sa zrýchlil približne o 20 %.

Zhrnutie

Profilovanie ukázalo, že dominantným bottleneckom je funkcia `pow`. Z toho vyplýva potenciálna optimalizácia kalkulačkovej knižnice, a to nahradenie `pow(x, 2)` jednoduchším násobením `x*x` (resp. `mul(x, x)`) v prípadoch druhej mocniny. Test tejto úpravy ukázal zníženie času behu približne o 20 %, čo potvrdzuje vhodnosť zamerať optimalizáciu práve na implementáciu funkcie `pow`.